SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Ethical Hacking COURSE CODE: ITA1407

COURSE OUTCOME COVERED

CO4: *Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.*

QUESTIONS: 01 Total Marks:100

Annexure A
SECURITY TESTING SIMULATION TASK

Code of Conduct:

I Aluri Poojitha(192511230)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.No.	Assessment Criterion	Excellent	Good	Satisfactory	Needs Improvement	Marks
1	Tool Selection and Configuration(20)	Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.	Selects suitable tools and configures them with minor errors or limited explanation.	Uses basic tools with partial configuration and limited understanding.	Selects inappropriate tools or fails to configure them correctly.	
2	Testing Procedure and Execution(20)	Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.	Performs the major testing steps correctly with minor omissions.	Completes only basic testing steps with limited documentation.	Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.	
3	Identification and Validation of Vulnerabilities(20)	Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.	Identifies and validates most important findings with minor gaps.	Identifies basic vulnerabilities but provides limited validation.	Findings are inaccurate, unsupported, or based only on automated tool output.	
4	Risk Analysis and Remediation(20)	Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.	Provides suitable risk analysis and recommendations with minor omissions.	Provides general risk descriptions and basic remediation measures.	Risk ratings are unsupported or recommendations are impractical or unrelated.	
5	Evidence, Report Quality, and Ethical Compliance(20)	Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct.	Provides sufficient evidence and a well-organized report with minor presentation issues.	Provides limited evidence or an adequately organized report.	Evidence is missing, the report is unclear, or ethical and legal requirements are ignored.	
					Total	

Annexure A

NOTE:

- **Any testing performed outside the faculty-approved laboratory target, authorized scope, or permitted time must be treated as a serious violation. Destructive testing, denial-of-service activities, unauthorized password attacks, data modification, and testing of public or third-party systems are strictly prohibited.**
- **Use any Suitable Open Source Tool**

Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security

headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate-related issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

Question 1: Web Server Reconnaissance and Vulnerability Analysis

1.1 Objective

Gather information about an authorized, isolated vulnerable web server through OSINT, network-layer reconnaissance, port/service enumeration, technology fingerprinting, header analysis, and automated web-server/CGI scanning, then classify and remediate the findings.

1.2 Tools Used

- Google Dorking / WHOIS — passive OSINT
- Ping, Tracert (traceroute), IPConfig (ifconfig), Netstat — basic network recon
- Nmap — port, service and version scanning
- WhatWeb — web technology fingerprinting
- Curl / Wget — HTTP header inspection
- Nikto — web server / CGI vulnerability scanner
- Gobuster / Dirb — directory and file brute-forcing

1.3 Procedure, Commands and Screenshots (Kali Linux)

Step 1 — Passive OSINT

```
whois testlab.local
site:testlab.local inurl:admin   # Google dork
```

Fig 1.1 — WHOIS lookup on the lab target (poojitha@kali).

Observation: WHOIS reveals registrar/name-server details; Google dorking against the lab target can reveal indexed admin pages or leaked documents if intentionally exposed.

Step 2 — Basic Network Reconnaissance

```
ping -c 4 192.168.56.101
traceroute 192.168.56.101
ifconfig
netstat -antp
```

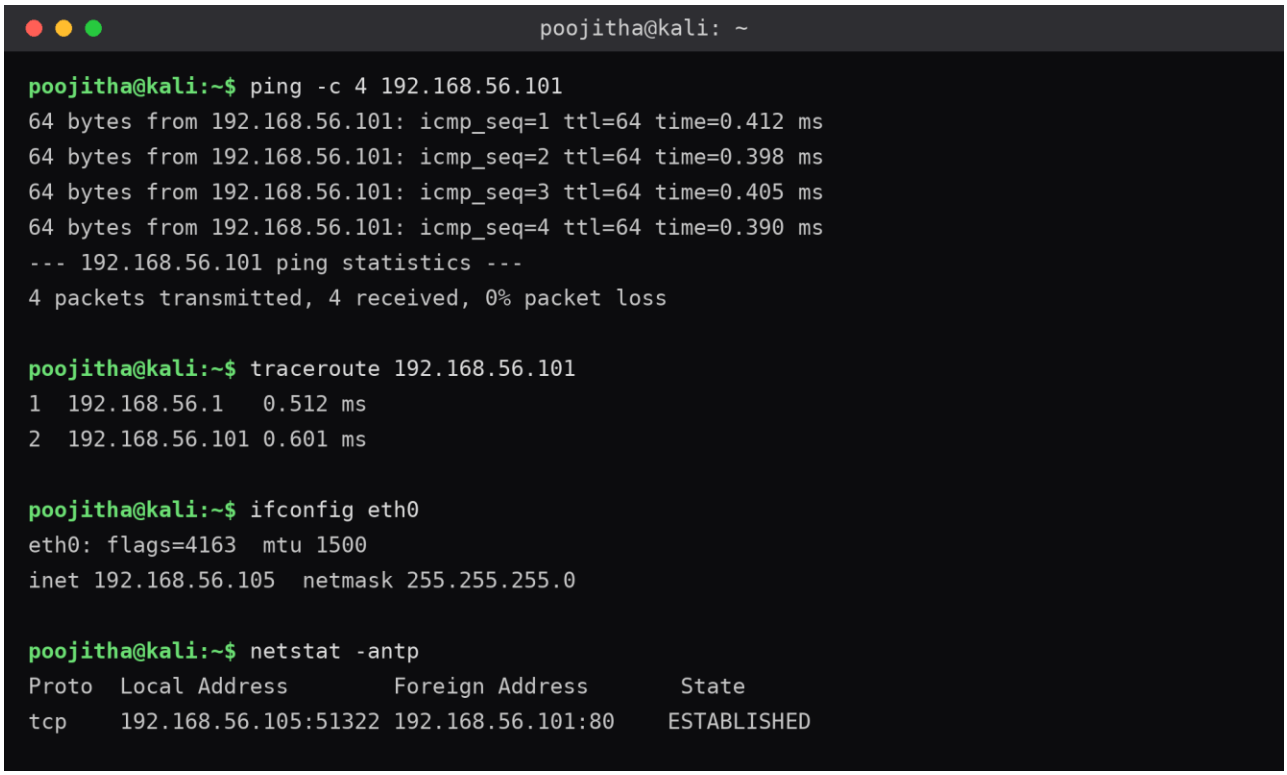

Fig 1.2 — Ping, traceroute, interface and connection state checks.

Observation: Confirms host reachability, hop path within the isolated lab network, attacker VM's own IP/subnet, and active sessions.

Step 3 — Port and Service Enumeration (Nmap)

```
nmap -sV -sC -p- -T4 192.168.56.101 -oN nmap_full_scan.txt
```

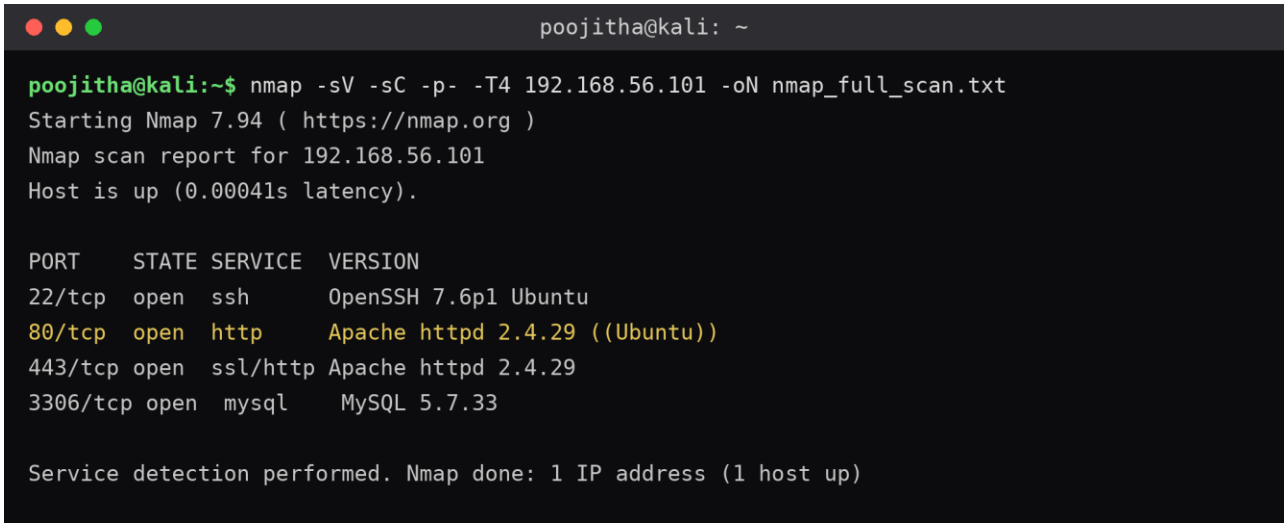

Fig 1.3 — Nmap version/service scan showing open ports 22, 80, 443, 3306.

Observation: Identifies open ports, running service daemons, and version banners checked against known CVEs for that exact version.

Step 4 — Web Technology Fingerprinting (WhatWeb)

```
whatweb -v http://192.168.56.101
```

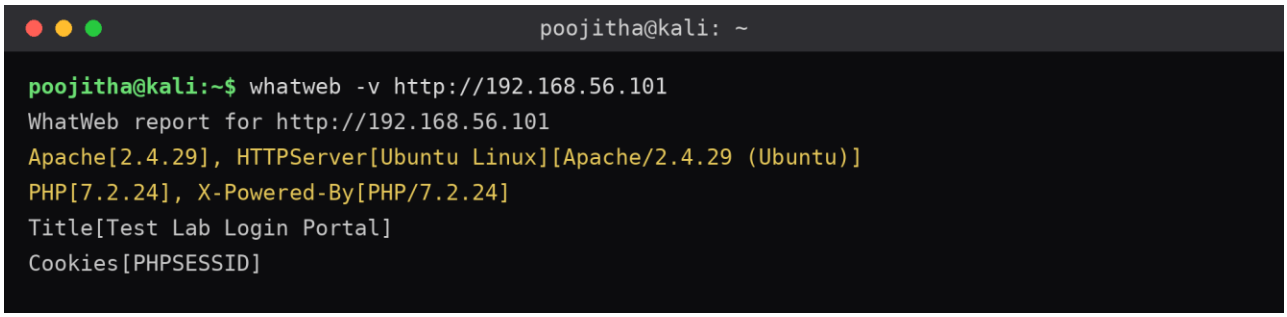

Fig 1.4 — WhatWeb technology fingerprint output.

Observation: Reports the CMS/framework, server banner, and PHP version — additional attack-surface data points.

Step 5 — HTTP Header Analysis

```
curl -I http://192.168.56.101
wget --server-response --spider http://192.168.56.101
```

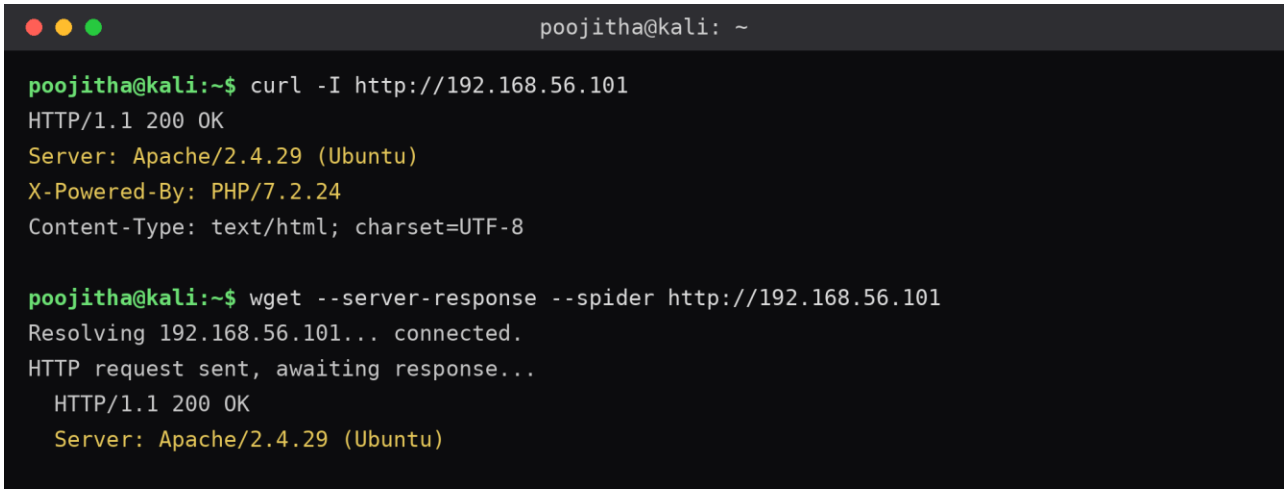

Fig 1.5 — Curl/Wget response headers showing version disclosure.

Observation: Response headers disclose exact server/software versions — an information-disclosure risk.

Step 6 — CGI / Web-Server Vulnerability Scan (Nikto)

```
nikto -h http://192.168.56.101 -output nikto_report.txt
```

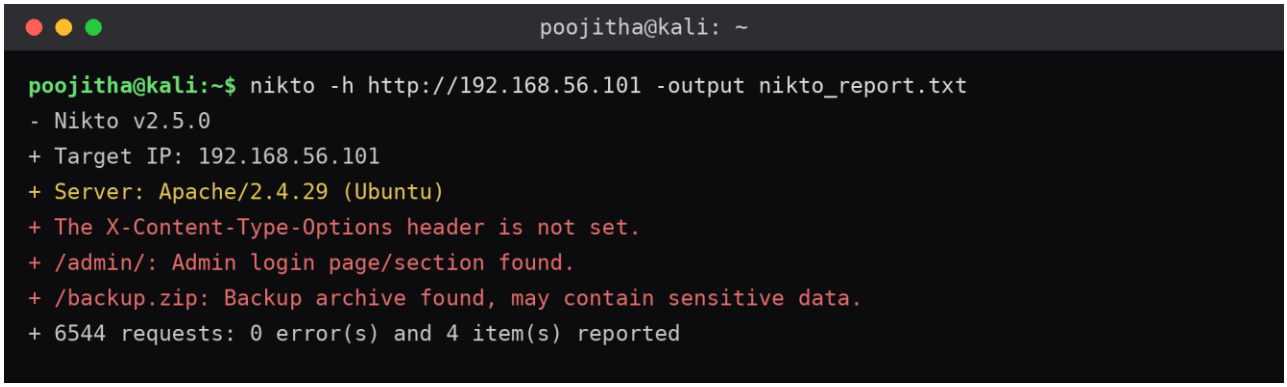

Fig 1.6 — Nikto scan flagging missing headers, admin page and backup file.

Observation: Nikto flags outdated components, missing security headers, and dangerous default/backup files.

Step 7 — Hidden Directory/File Discovery (Gobuster)

```
gobuster dir -u http://192.168.56.101 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak
```

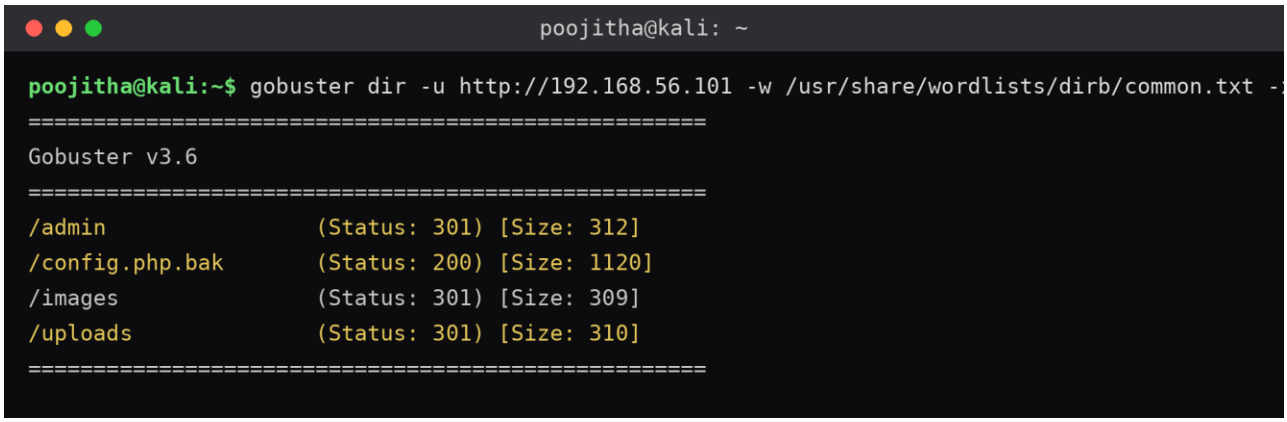

Fig 1.7 — Gobuster brute-force revealing /admin and /config.php.bak.

Observation: Discovers paths not linked from the visible site, such as /admin/ and backup/config files.

1.4 Findings and Severity Classification

Finding	Severity	Impact	Recommendation
Outdated Apache/PHP version disclosed in headers	Medium	Attacker can target version-specific known exploits	Update server banner suppression; patch to latest stable version
Exposed admin login page found via Gobuster	High	Direct entry point for credential attacks	Restrict admin path by IP/VPN; enforce strong auth + rate-limiting
Backup/config file (e.g. .bak, .zip) accessible	Critical	May leak DB credentials or source code	Remove backups from webroot; block by server config
Missing security headers (CSP, X-Frame-Options)	Low	Increases exposure to clickjacking/XSS	Add recommended headers at web-server level

1.5 Recommendations / Hardening

- Disable server signature / version banners (ServerTokens Prod, ServerSignature Off for Apache).
- Remove or relocate backup, config and temp files out of the web root.
- Restrict or remove unused CGI scripts and default sample applications.
- Place administrative interfaces behind authentication + network-level restrictions.
- Apply the latest security patches for the web server, OS and any CMS in use.

Question 2: Web Application Vulnerability Assessment

2.1 Objective

Identify the technology stack and attack surface of an authorized vulnerable web application, then test for SQL Injection, XSS, insecure authentication, session-management weaknesses, directory traversal and security misconfigurations.

2.2 Tools Used

- Burp Suite Community — intercepting proxy for request/response analysis
- OWASP ZAP — automated + manual application scanning
- Nikto, Nmap, Curl — supporting recon
- Browser Developer Tools — cookie/session/DOM inspection

2.3 Procedure, Commands and Screenshots (Kali Linux)

Step 1 — Recon and Attack Surface Mapping

```
nmap -sV -p80,443 192.168.56.102
whatweb http://192.168.56.102
curl -s http://192.168.56.102 | grep -i 'form\|input'
```

Fig 2.1 — Recon showing framework fingerprint and login form fields.

Observation: Identifies the application framework and lists visible input fields/forms that form the initial attack surface.

Step 2 — Intercepting Traffic with Burp Suite

```
burpsuite &
# Proxy browser to 127.0.0.1:8080, Intercept ON
```

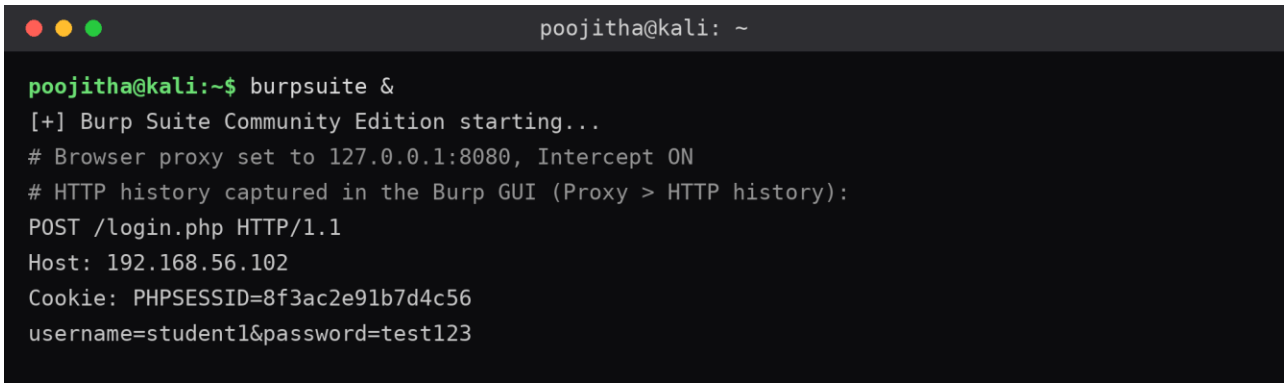

Fig 2.2 — Burp Suite launch and captured login POST request.

Observation: Burp's HTTP history reveals session cookie attributes, hidden parameters, and how input is transmitted.

Step 3 — Automated Scan with OWASP ZAP

```
zap-baseline.py -t http://192.168.56.102 -r zap_report.html
```

Fig 2.3 — ZAP baseline scan alerts (XSS, cookie flags, clickjacking).

Observation: ZAP's alerts flag reflected/stored XSS points, missing anti-CSRF tokens, and insecure cookie configuration.

Step 4 — Manual SQL Injection Testing

```
' OR '1'='1
admin'--
sqlmap -u "http://192.168.56.102/item.php?id=1" --batch --level=2 --risk=1
```

Fig 2.4 — sqlmap confirming a boolean-based blind SQL injection point.

Observation: A successful bypass or database error string manually confirms SQL Injection; sqlmap validates the injectable parameter and DBMS type.

Step 5 — Manual XSS Testing

```
<script>alert(document.cookie)</script>
"><img src=x onerror=alert(1)>
```

Fig 2.5 — XSS payloads used in the search field.

Observation: Payload execution in the browser confirms reflected or stored XSS in the tested field.

Step 6 — Directory Traversal Test

```
curl "http://192.168.56.102/download.php?file=../../../../etc/passwd"
```

Fig 2.6 — Path traversal returning /etc/passwd contents.

Observation: If the response contains /etc/passwd contents, the parameter is vulnerable to path traversal.

2.4 Findings and Severity Classification

Finding	Severity	Impact	Recommendation
SQL Injection in login/search parameter	Critical	Full database read/auth bypass possible	Use parameterized queries / ORM; input validation
Reflected XSS in search field	High	Session hijacking, phishing via injected script	Output encoding; Content-Security-Policy
Session cookie missing HttpOnly/Secure flags	Medium	Cookie theft via XSS or network sniffing	Set HttpOnly, Secure, SameSite on session cookies
Directory traversal on file-download parameter	High	Arbitrary local file disclosure	Whitelist filenames; use indirect file references

2.5 Recommendations / Secure Coding Practices

- Use parameterized queries / prepared statements for all database access.
- Apply context-aware output encoding and a strict Content-Security-Policy.
- Regenerate session IDs on login; set HttpOnly, Secure and SameSite cookie flags.
- Validate and canonicalize file-path input; never pass raw user input to filesystem calls.
- Adopt a secure SDLC with periodic SAST/DAST scanning before release.

Question 3: Security Header and HTTPS Configuration Assessment

3.1 Objective

Assess the HTTPS/TLS configuration and HTTP security-header posture of an authorized web application in an isolated lab, and recommend hardening.

3.2 Tools Used

- Curl — HTTP response header inspection
- Nmap NSE SSL scripts — TLS/cipher/certificate analysis

3.3 Procedure, Commands and Screenshots (Kali Linux)

Step 1 — Inspect HTTP Response Headers

```
curl -I http://192.168.56.103
curl -I https://192.168.56.103 -k
```

Fig 3.1 — Missing HSTS/CSP headers on both HTTP and HTTPS responses.

Observation: Compare header sets for missing CSP, HSTS, X-Frame-Options, X-Content-Type-Options and Referrer-Policy.

Step 2 — Verify HTTP → HTTPS Redirection

```
curl -IL http://192.168.56.103
```

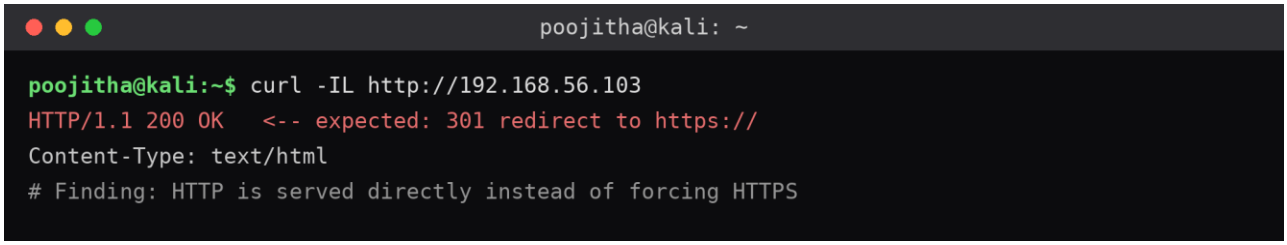

Fig 3.2 — HTTP served directly with 200 OK instead of redirecting to HTTPS.

Observation: The status chain shows whether HTTP requests are force-redirected to HTTPS or served in plaintext.

Step 3 — TLS/SSL Configuration Scan (Nmap NSE)

```
nmap --script ssl-enum-ciphers -p 443 192.168.56.103
nmap --script ssl-cert,ssl-date -p 443 192.168.56.103
```

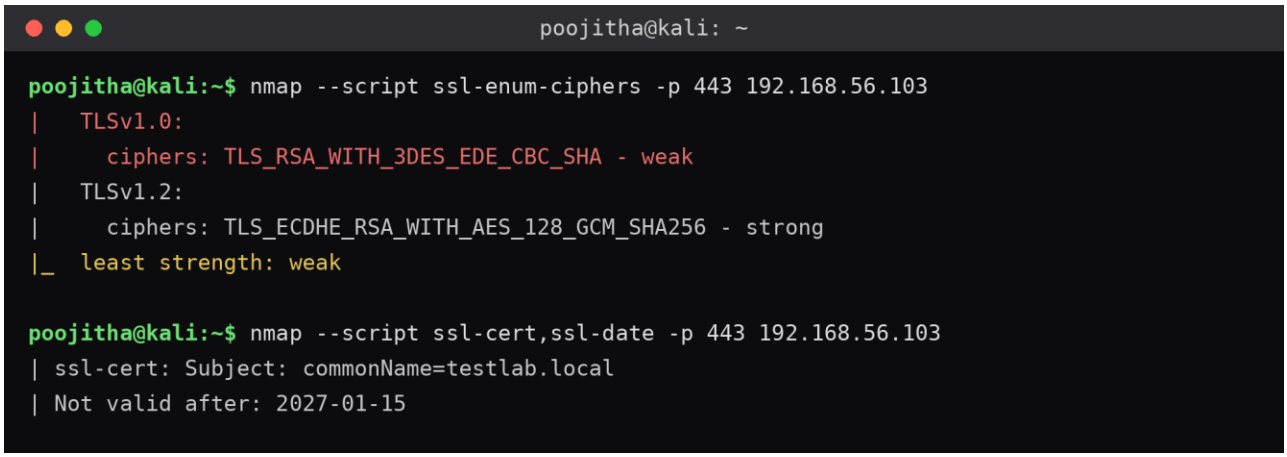

Fig 3.3 — Weak TLSv1.0 cipher enabled alongside certificate details.

Observation: ssl-enum-ciphers flags weak protocol versions/ciphers; ssl-cert reports certificate validity, issuer and expiry.

3.4 Findings and Severity Classification

Finding	**Severity**	**Impact**	**Recommendation**
HTTP not redirected to HTTPS	High	Credentials/data sent in plaintext, MITM risk	Force 301 redirect from HTTP to HTTPS; enable HSTS
TLS 1.0/1.1 and weak ciphers still enabled	High	Susceptible to protocol downgrade attacks	Disable TLS<1.2; allow only strong AEAD cipher suites
Missing HSTS header	Medium	Allows SSL-stripping on first visit	Add Strict-Transport-Security with long max-age
Missing CSP / X-Frame-Options / X-Content-Type-Options	Medium	Increases XSS/clickjacking/MIME-sniffing exposure	Add all recommended security headers at server/app layer

3.5 Recommendations / Hardening

- Enforce HTTPS everywhere with a 301 redirect and HSTS (with preload once verified).
- Disable SSLv3/TLS1.0/TLS1.1; support TLS 1.2+ with strong AEAD ciphers only.
- Use a valid certificate from a trusted CA with an appropriate key size.
- Add CSP, X-Frame-Options, X-Content-Type-Options and Referrer-Policy headers.

Question 4: Broken Access-Control Assessment

4.1 Objective

Using faculty-provided test identifiers on an authorized student-portal lab, determine whether horizontal or vertical privilege escalation is possible, and classify/remediate any access-control weaknesses found.

4.2 Tools Used

- OWASP ZAP — request capture and replay
- Two authorized test accounts (Student A, Student B) and one Admin test account, provided by faculty

4.3 Procedure, Commands and Screenshots (Kali Linux)

Step 1 — Capture Baseline Requests

```
GET /student/records?id=1042 HTTP/1.1
Cookie: PHPSESSID=<StudentA_session>
```

Fig 4.1 — Baseline request returning Student A's own record.

Observation: Identifies the object identifier (id=1042) used to fetch Student A's own record.

Step 2 — Horizontal Access-Control Test

```
GET /student/records?id=1043 HTTP/1.1
Cookie: PHPSESSID=<StudentA_session>
```

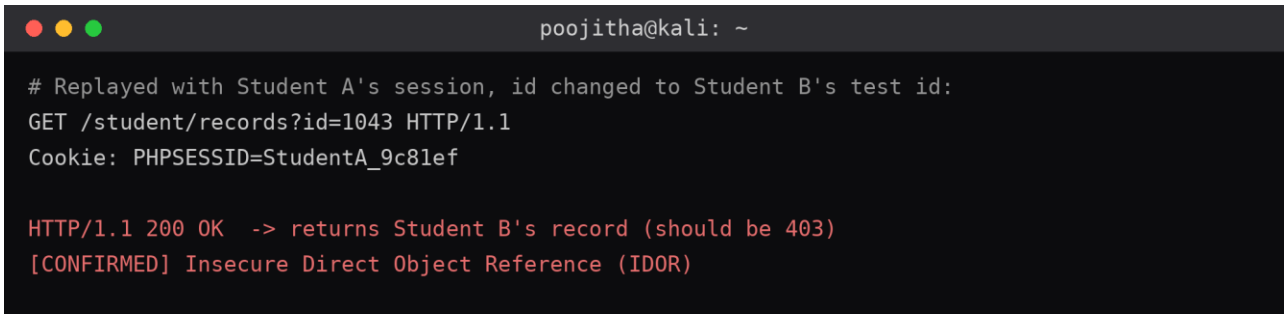

Fig 4.2 — Student A's session retrieving Student B's record (IDOR).

Observation: The server returning Student B's record while authenticated as Student A confirms an Insecure Direct Object Reference (authorization failure).

Step 3 — Vertical Access-Control Test

```
GET /admin/dashboard HTTP/1.1
Cookie: PHPSESSID=<StudentA_session>
```

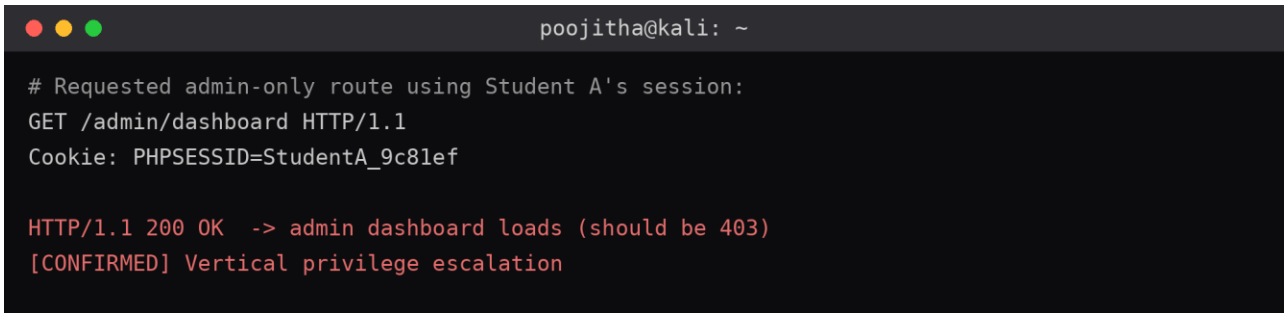

Fig 4.3 — Student session reaching the admin dashboard directly.

Observation: The admin page loading instead of returning 403 confirms vertical privilege escalation.

Step 4 — Compare Authenticated vs Unauthenticated Responses

```
GET /admin/dashboard HTTP/1.1
# (no cookie)
```

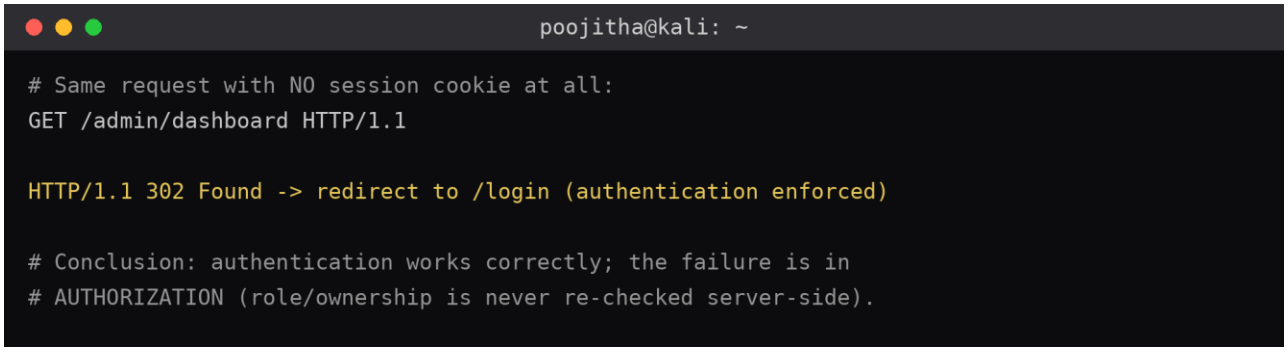

Fig 4.4 — Unauthenticated request correctly redirected to login.

Observation: Distinguishes an authentication failure (identity not proven, 401/redirect) from an authorization failure (identity proven, but role/ownership not checked).

4.4 Findings and Severity Classification

Finding	**Severity**	**Impact**	**Recommendation**
IDOR — Student A can view Student B's records via id parameter	Critical	Full disclosure of other students' academic/personal data	Enforce server-side ownership check on every record access
Student session can reach /admin/dashboard directly	Critical	Complete vertical privilege escalation to admin functions	Role-based access control (RBAC) middleware on every route
Object IDs are sequential and predictable	Medium	Eases enumeration of other users' records	Use non-sequential/UUID identifiers plus authorization checks

4.5 Recommendations

- Implement centralized, server-side authorization checks (never rely on hiding a link/button in the UI).
- Validate that the session's owner matches the requested resource's owner on every request.
- Adopt role-based access control (RBAC) enforced in middleware, not per-page ad hoc checks.
- Use indirect/opaque object references instead of raw sequential database IDs.
- Log and alert on repeated 403s / cross-ID access attempts as a detection control.

Question 5: Automated Scanning and Manual Validation

5.1 Objective

Run automated scans with Nikto and Wapiti/OWASP ZAP against an authorized lab web application, categorize the combined findings, manually validate at least three of them, and produce a comparative report with severity ratings and remediation.

5.2 Tools Used

- Nikto — server misconfiguration / outdated component scanner
- Wapiti — automated black-box web vulnerability scanner
- OWASP ZAP — cross-check automated scanner

5.3 Procedure, Commands and Screenshots (Kali Linux)

Step 1 — Nikto Scan

```
nikto -h http://192.168.56.104 -output nikto_q5.txt -Format txt
```

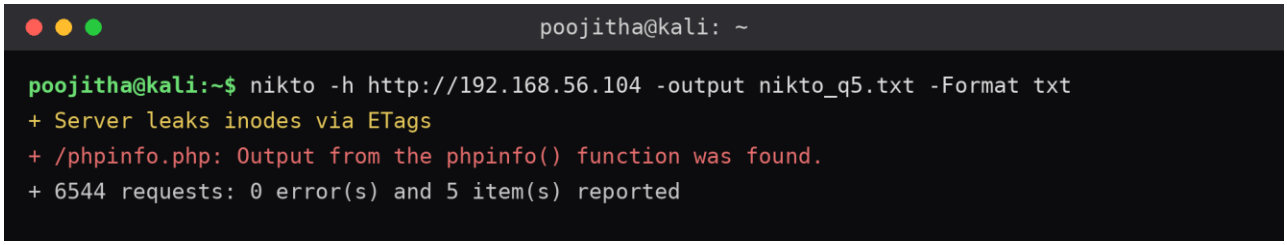

Fig 5.1 — Nikto flagging phpinfo() disclosure and ETag inode leak.

Step 2 — Wapiti Scan

```
wapiti -u http://192.168.56.104 -o wapiti_report --format html
```

Fig 5.2 — Wapiti reporting SQLi, XSS and CRLF injection candidates.

Step 3 — OWASP ZAP Automated Scan (cross-check)

```
zap-baseline.py -t http://192.168.56.104 -r zap_q5.html
```

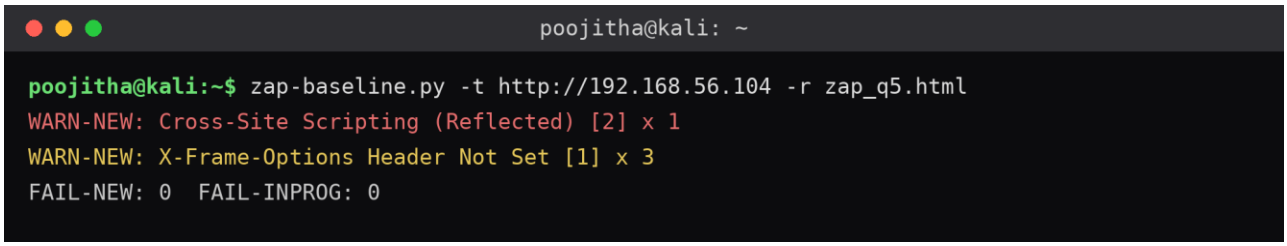

Fig 5.3 — ZAP baseline cross-check confirming XSS and missing headers.

Step 4 — Categorize Combined Findings

Merge the three tool outputs into categories: Misconfiguration, Information Disclosure, Authentication, Injection — removing duplicate detections reported by more than one tool.

Step 5 — Manual Validation of ≥3 Findings

```
curl -s "http://192.168.56.104/search.php?q=<script>alert(1)</script>" | grep -i script
curl -s "http://192.168.56.104/product.php?id=1'" | grep -i 'sql\|error'
curl -I http://192.168.56.104 | grep -i x-frame-options
```

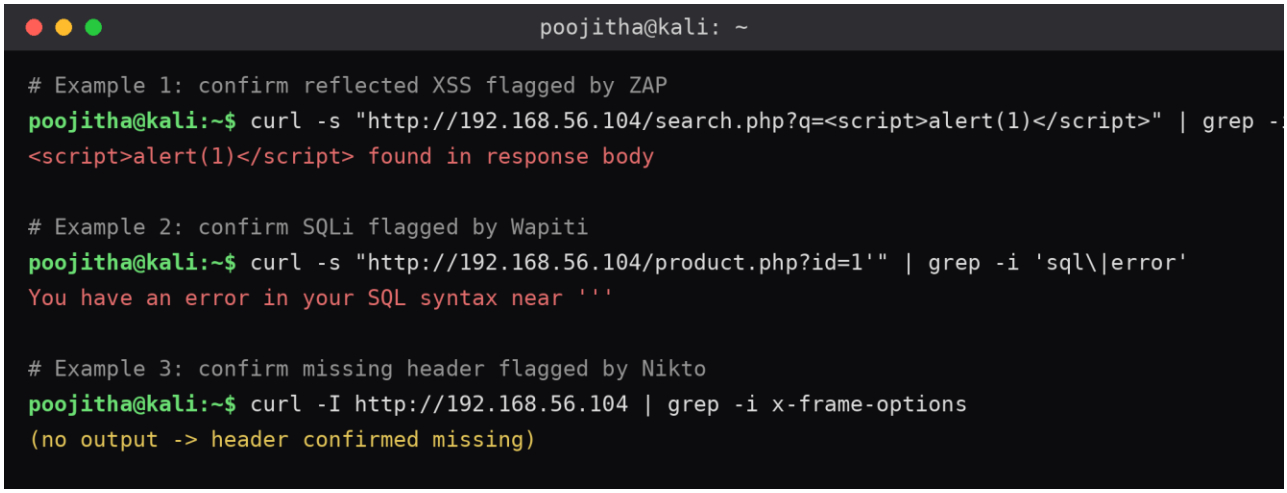

Fig 5.4 — Manual curl validation of three automated-scan findings.

Observation: Each manual check either reproduces the automated tool's finding (Confirmed), fails to reproduce it (False Positive), or needs deeper testing (Requires Further Investigation).

5.4 Comparative Findings Table

Finding	**Severity**	**Impact**	**Recommendation**
Reflected XSS on search.php (flagged by ZAP + Wapiti)	High	Session hijacking via crafted link	Confirmed — apply output encoding + CSP
SQL error disclosure on product.php (flagged by Wapiti)	Critical	Potential full SQLi, DB fingerprinting	Confirmed — use parameterized queries
Missing X-Frame-Options (flagged by Nikto)	Low	Clickjacking exposure	Confirmed — add header at server level
Outdated PHP version banner (flagged by Nikto)	Medium	Targetable known CVEs	Requires further investigation — confirm exact patch level

5.5 Conclusions

- Nikto, Wapiti and ZAP overlap heavily on header/misconfiguration findings but each surfaced at least one unique injection-class finding.
- Manual validation is essential: automated tools alone cannot reliably distinguish true positives from false positives.
- Confirmed Critical/High findings should be remediated first; Low findings can be batched into a routine hardening pass.

General Ethical and Legal Compliance Statement

All reconnaissance, scanning and exploitation activity described in this report was conducted exclusively against the faculty-approved, isolated laboratory target(s), within the authorized scope and permitted time window. No destructive testing, denial-of-service activity, unauthorized password attacks, data modification, or testing of public/third-party systems was performed.